

```
$ cat notes.md
```

DevOps & Cloud [SWE] — Class 10 Notes

LECTURE TITLE Kubernetes Pods, ReplicaSets & Deployments

DATE/TIME 3 September 2026, 2:00 PM

DURATION 120 minutes

INSTRUCTOR Nensi Ravaliya

BIG IDEA Every Kubernetes file is the same four fields. Learn the shape once, and every object you ever write reads the same way.

SOURCES this class's transcript + `pod.yml` / `test.yml` (written by you in class) + her **Kubernetes repo** (`architecture.md`, `core-objects.md`) and **devops-heros** → `session10-k8s-core-objects/`

WHAT'S INSIDE — 17 SECTIONS

- 00 ⚡ The 60-Second Version
- 01 📅 Housekeeping
- 02 🏆 Setup Lab — Get A Cluster Running First
- 03 💬 Architecture Recap — The Six Components
- 04 🚶 The Journey of kubectl apply — Frame by Frame
- 05 💎 The Controllers — More Than One
- 06 🌱 Pod Anatomy — The Four Facts
- 07 📦 The Hierarchy — Container → Pod → ReplicaSet → Deployment
- 08 💎 The Four Fields — In Every File, Forever
- 09 🚦 Ports — Different From Docker
- 10 🏆 Lab 1 — Deploy Your First Pod
- 11 🏁 The Pod Lifecycle — And The Three-Stage Race
- 12 🏠 minikube vs kind vs kubectl vs EKS
- 13 📄 The Command Set
- 14 📝 Homework
- 15 🧠 Cheat-Sheet
- 16 📄 Notes on the Transcript Itself

Rebuilt from the raw class transcript — nothing cut, just organised, de-noised and made readable. theme: Kube Indigo

00



The 60-Second Version

1. **Everything in Kubernetes is an API object.** That's why line 1 of every file is `apiVersion` . Delete the object, the real thing disappears.
2. **Four fields, every single time:** `apiVersion` · `kind` · `metadata` · `spec` . There are no exceptions.
3. **The six components never talk to each other.** etcd, scheduler, controller and kubelet all speak *only* through the **API server**. This is the single most asked architecture question.
4. **Pod = smallest deployable unit.** Containers inside one pod **share a network and a volume** — so they reach each other on `localhost` .
5. **Hierarchy:** Container → **Pod** → **ReplicaSet** → **Deployment**. Each layer wraps the one below.
6. `kubectl apply -f file.yaml` replaces the whole `docker run -dit -p ...` dance.
7. **A pod with no long-running process finishes and goes `Completed`** . nginx runs forever; busybox with a `command` completes. Same rule as Docker's PID 1.
8. `ErrImagePull` means you typo'd the image name. The pod still gets created — it just can't start.



01



Housekeeping



She made her personal Kubernetes repo public



SAID IN CLASS

"This one I added. It was private, I told you. So I made it public today. It's basically my personal Kubernetes repo that I am using for any interview till now."



github.com/Nency-Ravaliya/Kubernetes

File	What's in it
<code>architecture.md</code>	control plane + worker node, every component
<code>core-objects.md</code>	Pod, ReplicaSet, Deployment, Service, Namespace — with YAML

SAID IN CLASS

"I created this long back — maybe one year, two years, three years. But this repo actually helped me in every interview. Whatever interview I'm giving, I just read my own repo. Not even the Kubernetes documentation — because I know, somewhere I get stuck, then I check. But this is pretty enough for you guys until you have four years of experience in DevOps."

"Documentation is too boring, right?"

She polled the room on how they'd actually studied — official docs? GeeksforGeeks? Most said **ChatGPT**.

SAID IN CLASS

"Documentation is too technical, I know. When you read it for the first time it is too technical. After two years of experience in DevOps you feel like — yeah, yeah, I can read. After four years you're like, okay, this is the only thing that I want. It's a process."

LOCK THIS IN

Don't feel bad about this. Official docs are written as a *reference*, not a *tutorial* — they assume you already know the vocabulary. Using an LLM or a curated repo to get the mental model first, then reading the docs to get the detail right, is a completely legitimate order of operations.

She asked what format you want

SAID IN CLASS

"Do you need a PPT at the end of the semester, or a readme.md file? Let me know."

Her reasoning: *"Just teaching here or doing hands-on is one type of learning. But it's a bit hard for you guys to remember the last thing plus new things — if we're completing Kubernetes in one week, and we already completed Docker in just three lectures. So you need a readme.md whatever we learn every day."*

Her plan: a README per session → then generate a **PPT** from it → possibly a **GitBook**. *"No much technical documentation, really a simple way, just like I'm teaching you — just the main point of the topic. Like: kube-apiserver, we call it the gatekeeper, front door of our cluster. Only three points with an example."*

The minikube ultimatum

She asked three separate times who had installed **minikube**, then:

 CLASS MOMENT

"I will come today to your desk and I will check whether you have minikube or not."

And at the end of class: *"Next lecture, I want minikube on each and every PC. If it is not there — get out, honestly."* 

 *"Kubernetes is the most important concept in DevOps. If you don't know Kubernetes, then nothing — you will be washed out of DevOps itself."*



02



Setup Lab — Get A Cluster Running First

Nothing else in this class works until this does. Do it now.



DO THIS YOURSELF

Lab 0 — install and start a local cluster.

Step 1 — install minikube (pick your OS from minikube.sigs.k8s.io/docs/start):

```
# macOS
brew install minikube

# Windows (PowerShell as admin)
choco install minikube

# Linux
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

Step 2 — start it. (Docker Desktop must already be running — minikube uses it as its driver.)

```
minikube start
```

Step 3 — confirm, three ways:

```
minikube status      # → host: Running, kubelet: Running, apiserver: Running
kubectl version      # → Client Version + Server Version
kubectl cluster-info  # → "Kubernetes control plane is running at https://
127.0.0.1:..."
kubectl get nodes     # → one node, STATUS Ready
```

✅ You're ready when `kubectl get nodes` shows a node with **STATUS: Ready** .

Her fallback ladder when minikube won't install

SAID IN CLASS

"If minikube is not working, then what you can do:"

- 
1. minikube ← try this first
↓ fails
 2. kind ← "Kubernetes IN Docker", also a local cluster
↓ fails
 3. Docker Desktop ← it has a built-in Kubernetes toggle in Settings
↓ fails
 4. A cloud provider ← spin up an EC2 instance and install there

 "These are the ways. You've been stalled here — then how are you proceeding with further steps?"

The #1 error in the room: "resource not found"

Two students hit it. Her fix was refreshingly blunt:

WATCH OUT

If `kubectl get nodes` says *resource not found*, or you can't see your control plane and worker node — open Docker Desktop.

"That's it. Nothing else. In Docker Desktop you will see these two containers: `demo-cluster-control-plane` and `demo-cluster-worker`. Once you open your Docker Desktop, your major issue will be vanished, surely."

 **Why this actually works:** minikube and kind don't run a real machine — they run your "cluster" as **Docker containers on your laptop**. If the Docker engine isn't up, there is literally nothing for `kubectl` to talk to. Opening Docker Desktop starts the engine, the cluster containers come back, and `kubectl` finds its server again.

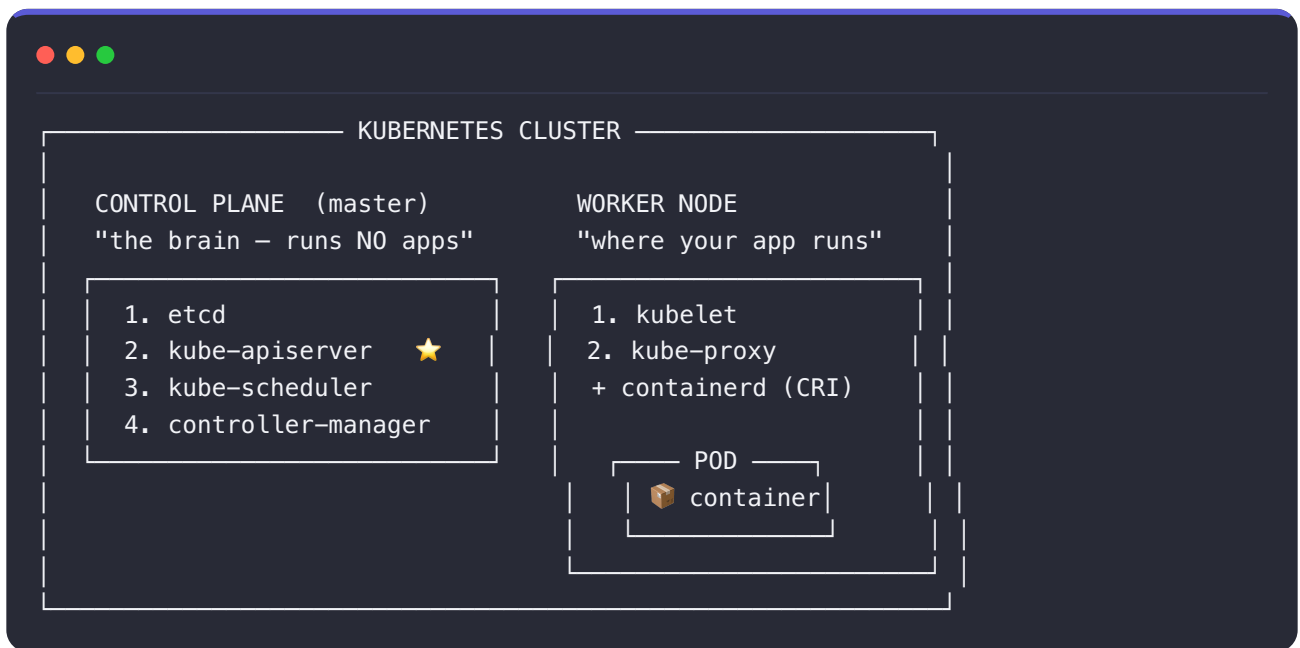


03



Architecture Recap — The Six Components

She had the class read `architecture.md` for five minutes, then quizzed them.



SAID IN CLASS

"Master node, worker node. I'm going to use master and worker because it's my favourite name. That's why. Nothing else." 🙄

🙄 And on the official diagram: "Only two colours are there. They are not making it colourful even."

What each one does, in one line

Component	One-line job
etcd	the database — key-value, stores the entire cluster state
kube-apiserver	★ the gatekeeper / front door — everything goes through it
kube-scheduler	decides which node a new pod lands on
controller-manager	keeps reality matching what you asked for
kubelet	on each node: creates/runs/deletes containers + sends the heartbeat
kube-proxy	networking and network policies on that node
containerd (CRI)	the thing that actually runs the container

💎 Everything is an API object

🗣️ SAID IN CLASS

"In Kubernetes, everything we create is an API object. The file I'm creating — it's an API object."

Look at the top line of any file and you'll see it:

```
Pod      → apiVersion: v1
Service  → apiVersion: v1
ReplicaSet → apiVersion: apps/v1
Deployment → apiVersion: apps/v1
StatefulSet → apiVersion: apps/v1
```

✅ **And the consequence she drew out:** *"If I delete that API object, my actual deployment or Kubernetes object will be deleted."* The object is the thing. There's no separate "real" resource hiding behind it.

🧩 WHY IT WORKS

Why `v1` for some and `apps/v1` for others. Kubernetes groups its API into families. The **core group** (the original objects — Pod, Service, Namespace, ConfigMap, Secret) has no prefix, so it's just `v1`. Anything that **manages pods for you** lives in the **apps** group — ReplicaSet, Deployment, StatefulSet, DaemonSet — so it's `apps/v1`.

The shortcut: *if it runs a container directly, it's `v1`. If it babysits something that runs containers, it's `apps/v1`.*



04 🎬 The Journey of `kubectl apply` — Frame by Frame

This was the heart of the lecture, and it's the question she says gets asked in every interview. She narrated the whole loop; here it is as a sequence.



WATCH IT HAPPEN

You type one command. Here's every frame of what happens next.



FRAME 1

YOU: `$ kubectl apply -f deployment.yaml`

|
| an API CALL
▼

API SERVER

 ← the ONLY door in

FRAME 2 — validate

API SERVER: "is this YAML even legal?"

✓ valid → carry on

✗ invalid → error straight back to your terminal

FRAME 3 — store

API SERVER → etcd: "save this new service"

- name
- it's a pod
- needs 20 GB memory, 16 core CPU
- taints / tolerations / affinity

etcd: "stored." ✓

FRAME 4 — the scheduler's polling loop

every ~3 seconds:

SCHEDULER → API SERVER: "any new service?"

API SERVER → etcd: "anything new?"

etcd → API SERVER: "yes – here are the details"

API SERVER → SCHEDULER: "yes – here are the details"

FRAME 5 — shopping for a node

Needs: 40 GB memory, 16 cores

- node 1 → 100 services running, only 5 GB free ✗ no
- node 2 → 16 cores ✓ but only 20 GB memory ✗ no
- node 3 → 50 GB memory, 20 cores ✓ YES

SCHEDULER: "node 3, please take this pod."

FRAME 6 — the controller takes over

CONTROLLER: "the spec says 3 replicas. How many exist?"

→ API SERVER → kubelet reports back

FRAME 7 — kubelet does the actual work

KUBELET on node 3: pulls the image

starts the container

💖 heartbeat → API SERVER: "1/1 Running"

FRAME 8

`$ kubectl get pods`

NAME	READY	STATUS	AGE
nginx-pod	1/1	Running	12s



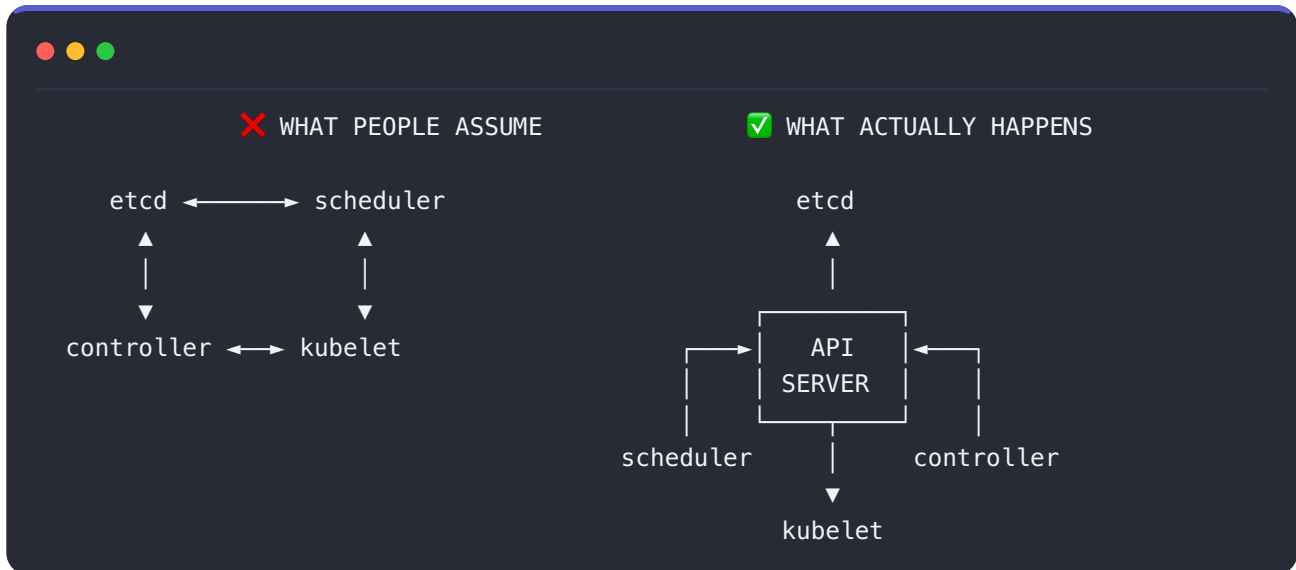
🔑 The golden rule — and the question they always ask

🎯 INTERVIEW ALERT

She stopped and made this a whole segment:

"Can you see these six components cannot talk with each other? etcd cannot directly talk with the scheduler. Scheduler cannot talk with the controller. Controller cannot talk with kubelet."

"Why do we need the API server? The controller knows it needs three replicas — why can't the controller directly tell kubelet 'hey, I need three replicas, can you create it?' No. It is not going to."



✅ LOCK THIS IN

Every arrow in Kubernetes points at the API server. Want to ask the scheduler something? You ask the API server, it asks the component, takes the result, and hands it back.

🧩 WHY IT WORKS

Why it's built this deliberately awkward way — she didn't cover the reasoning, and it's worth having for the follow-up question:

Benefit	What it buys
One place for security	authentication, authorisation and audit logging happen once, not six times
One place for validation	a malformed object is rejected at the door
Loose coupling	you can swap the scheduler for a custom one and nothing else notices
One audit trail	every change to the cluster is a request through one component

🧠 **It's a call centre, not a group chat.** Nobody has anyone else's direct number. Everyone rings the switchboard, and the switchboard keeps the recording.

**05**

◆ The Controllers — More Than One

SAID IN CLASS

"Someone needs to manage this stuff, right? We created three frontend services. These three containers are running. Now what if one container fails? Who checks on it? Who is going to create a new one?"

ReplicaSet controller

LOCK THIS IN

Job: maintain the desired number of replicas.

```
YOU SAID:    3 nginx pods
REALITY:     2 nginx pods      ← one crashed
              ↓
CONTROLLER:  "that's one short"
              ↓ via API server
KUBELET:     creates pod #3     back to 3
```

 *"Replica — what do you mean by the word replica? Copy. So if I need three nginx containers and one is down, the controller will take care of it."*

Node controller — the one most people forget

She drew this out because it answers a failure the ReplicaSet controller *can't* see.

INTERVIEW ALERT

The scenario: *"My service is up and running right now, but I'm not able to reach it. I checked — service is up and running, but still I can't reach it. Maybe my server is down."*

What could have happened to the node itself:

```
x server rebooted
x server IP changed
x DNS configuration not working
x port not working
x security group blocking it
```

 *"Node equals your server. A server has an IP, a server has DNS, a server has its security groups."*

✅ **So the node controller's job is to check the machine itself is healthy** — and if not, raise an alert through the API server. Then a **Kubernetes administrator** creates a new server manually and attaches it to the cluster.

PICTURE THIS

Two controllers, two different questions:

Controller	Asks
ReplicaSet controller	<i>"are there enough pods?"</i>
Node controller	<i>"are the machines even alive?"</i>

A ReplicaSet controller on a dead node can't fix anything — it's on the dead node. That's why the node controller exists, in the control plane, watching from outside.

What kubelet actually does

She described it two ways, and both are true:

✅ LOCK THIS IN

Job 1 — the heartbeat. *"Kubelet's job is only sending the heartbeat of containers to the API server. Maybe in milliseconds, microseconds, even nanoseconds: 'my pod is running, three pods are running inside node 1, four pods are running in node 2.'"*

✅ **Job 2 — doing the work.** *"Kubelet is the one who is actually running your application — creating a pod, deleting a pod, stopping your pod."*

kube-proxy and CRI

NOTE

kube-proxy is technically optional *"just like the cloud controller manager. But by default we are using it. In any organisation, kube-proxy is there."* It handles all the networking inside a node.

WHY IT WORKS

CRI — why an "interface" at all? *"In which interface will my container run? I need something to run my container. Either Docker, either containerd, or any other open-source tool."*

CRI = Container Runtime Interface. It's a **plug socket**, not a plug. Kubernetes defines the shape of the socket; any runtime that fits can be plugged in.

```
Kubernetes → [ CRI socket ] → containerd ← the default today
                                   → CRI-O      ← Red Hat's
                                   → Docker      ← removed in K8s 1.24 (2022)
```

 Her history: *"Previously when Kubernetes was invented we used Docker as the container runtime. Then after some version, Docker was not supported. Now no one is using Docker as the container runtime. We use containerd."*

 **Worth one clarification:** this sounds like Docker was rejected, and that's not quite it. `containerd` was **pulled out of Docker itself** — Docker uses it internally to this day. Kubernetes used to reach Docker through an extra translation layer called *dockershim*, and in **v1.24** it deleted that layer and started calling `containerd` directly. **Your images are completely unaffected** — same image format either way. Kubernetes just cut out the middleman.

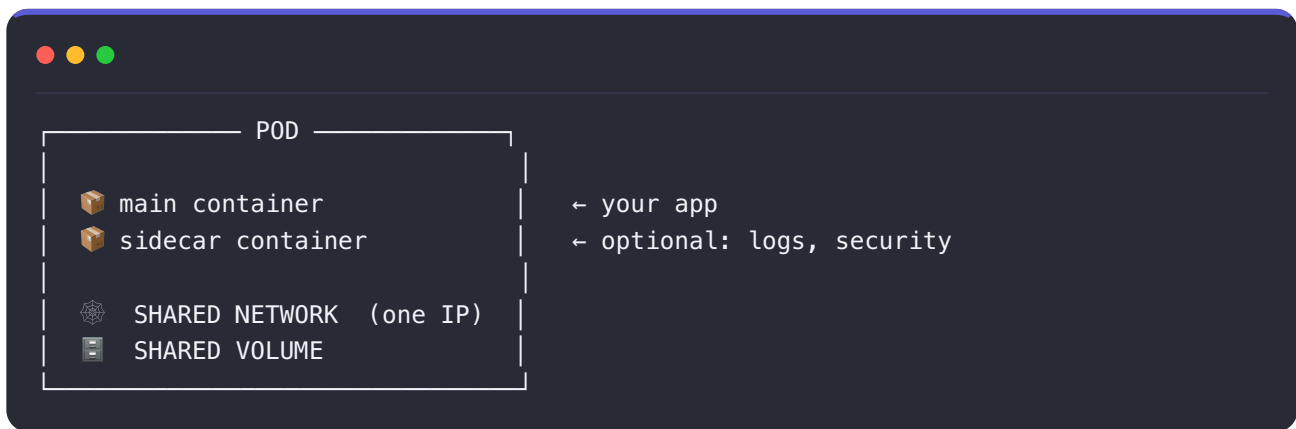


06

Pod Anatomy — The Four Facts

SAID IN CLASS

"I'm writing this because it's very important, that's why."



#	Fact
1	Pod is the smallest deployment unit in Kubernetes
2	Contains one main container — sometimes more
3	All containers in a pod share the same network
4	All containers in a pod share volumes

🧩 Why containers in a pod share a network

She anticipated the obvious objection — *"in Docker we made separate networks to isolate things, why not here?"*

🗣️ SAID IN CLASS

"Why do we need a different network here? Because we are running backend and DB in other pods. It's already separated. I don't need to separate it with another network."

✅ Exactly right, and here's the mechanical version:

DOCKER (Class 8)	KUBERNETES						
separate networks to keep frontend away from DB	separation happens at the POD boundary						
	<table><tr><td>pod A: frontend</td><td> different</td></tr><tr><td>pod B: backend</td><td> pods,</td></tr><tr><td>pod C: database</td><td> different IPs</td></tr></table>	pod A: frontend	different	pod B: backend	pods,	pod C: database	different IPs
pod A: frontend	different						
pod B: backend	pods,						
pod C: database	different IPs						
	inside ONE pod → same IP, so container A reaches container B at → localhost:8080 ✅ no networking needed						

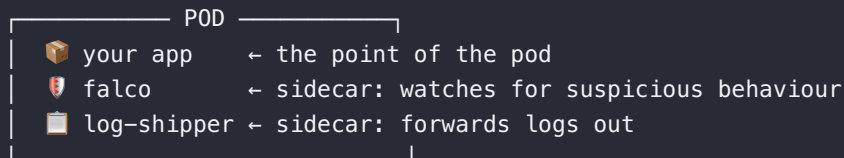
So the pod *is* the isolation boundary. Things inside it are meant to be intimate; things that need separating go in different pods.

🛡️ Sidecar / agent containers — with a real tool

🗣️ SAID IN CLASS

"What do you mean by agent or sidecar container? Let's suppose I'm deploying one service, and for that service I need to add a security tool in each node."

Her example: Falco — "Falco is a security tool, and we're using Falco to manage the security inside your running container."



✅ "Most of the time there is only one container to run your main service. Other containers may be a log container just to catch the logs, or an agent container adding security layers."

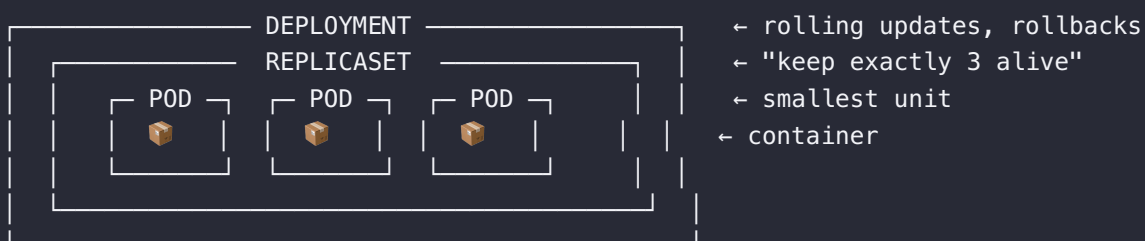
📌 And she flagged the object that puts one of these on **every** node: a **DaemonSet**. "DaemonSet means I want to add one security container in each node." That's your homework.



07 📦 The Hierarchy — Container → Pod → ReplicaSet → Deployment

🗣️ SAID IN CLASS

"Previously we had the Docker container. Right now we have a pod. Same thing — a Docker container equals a pod, you can say."



✓ LOCK THIS IN

Each layer adds exactly one capability:

Layer	Adds
Container	runs your code
Pod	a shared network + volume, scheduled as one unit
ReplicaSet	"there must always be N of these"
Deployment	"and here's how to change the version without downtime"



08 💎 The Four Fields — In Every File, Forever

🗣️ SAID IN CLASS

"Any YAML file you are taking for Kubernetes — apiVersion will be there, kind will be there, metadata and spec. Four words. Any Kubernetes file you create, you need to add these four things."

```
apiVersion: v1      # 1. WHICH kind of API object, and which version
kind: Pod           # 2. WHAT am I creating
metadata:          # 3. DATA ABOUT the thing — mainly its name
  name: nginx-pod
spec:              # 4. THE ACTUAL CONFIGURATION
  containers:
    - name: nginx-container
      image: nginx:latest
      ports:
        - containerPort: 80
```

Field	Her explanation
apiVersion	"Which type of API object it is, which version we are using." Taken from the official docs.
kind	"Which core object." Pod / Service / ReplicaSet / StatefulSet / Deployment
metadata	"Data about data — right? So here it's data about the pod." Mainly the name .
spec	"Here we are adding the actual configuration of what we're deploying."

Her naming conventions

GOOD TO KNOW

"It's a standard way" — suffix the name with what the object is:

Object	Name it
Pod	nginx-pod
ReplicaSet	myapp-rs
Service	myapp-service
Deployment	myapp

⚠️ **And note the trap she pointed out:** "Container name is different and pod name is different, by the way." In her file the pod is `nginx-pod` and the container inside it is `nginx-container`. Two separate names, two separate `name:` fields.

⚠️ YAML is case-sensitive

⚠️ WATCH OUT

"Whatever keywords we're writing — it's case sensitive. You cannot write `apiversion` in small case."

```
apiVersion: v1      # ✓
apiversion: v1      # ✗ error
APIVersion: v1      # ✗ error
```

🔧 "If there is a typo, or maybe some uppercase/lowercase issue, then when you do `kubectl apply -f` you will face an issue."



09 🚦 Ports — Different From Docker

🔧 SAID IN CLASS

"In Docker we add a port like `-p 80:80`. But here we're writing `containerPort`, `nodePort` and `hostPort`."

	Docker	Kubernetes
Inside the container	—	<code>containerPort: 80</code>

	Docker	Kubernetes
On the node	<code>-p 8080:80</code>	<code>nodePort: 30080</code> (on a Service)
Where it's written	the <code>docker run</code> command	the YAML file

✓ LOCK THIS IN

For today you only need `containerPort`. *"I don't need to specify two or three ports unless I want to expose my application outside of the cluster."*

🧠 PICTURE THIS

The mental model she gave:



INSIDE the cluster you can reach any pod. `containerPort` is enough.

OUTSIDE the cluster you need a SERVICE + `nodePort`
 → "server IP : `nodePort`"

🗣️ *"In an organisation they're using a server, and the cluster is running inside that server, so they can access that service. But if they want to access it outside the cluster, then they create a Service and mention a `nodePort`."*

📌 That's Class 11's topic. Today, everything stays inside.



10 🏆 Lab 1 — Deploy Your First Pod

🗣️ SAID IN CLASS

Write it, don't copy it. *"I already added this file in the DevOps Heroes repo, but you need to write it. Why? When I was learning Kubernetes, my teacher told me: you're going to write it. Then and then you'll get to know `apiVersion` is there, `metadata` is there. Otherwise it's like you're copy-pasting. You need to write it at least once."*

DO THIS YOURSELF

Lab 1 — nginx in a pod.

Step 1 — type this out by hand into `pod.yml` :

```
apiVersion: v1
kind: Pod

metadata:
  name: nginx-pod
  labels:
    app: nginx

spec:
  containers:
  - name: nginx-container
    image: nginx:latest
    ports:
      - containerPort: 80
```

Step 2 — deploy it:

```
kubectl apply -f pod.yml
# → pod/nginx-pod created
```

Step 3 — check it (this is your `docker ps`):

```
kubectl get pods
# NAME          READY   STATUS    RESTARTS   AGE
# nginx-pod     1/1     Running   0           12s
```

✅ **READY 1/1** is the win condition. One container wanted, one container ready.

Step 4 — read the logs (your `docker logs`):

```
kubectl logs nginx-pod
```

Step 5 — see the hidden detail:

```
kubectl get pods -o wide
# → adds IP and NODE columns
```

Step 6 — clean up:

```
kubectl delete pod nginx-pod
kubectl get pods      # → No resources found
```

What `-o wide` reveals

SAID IN CLASS

"What else can you see here? Your IP address — for that particular pod. Then which node it is deployed on."

```
NAME      READY   STATUS    IP           NODE
nginx-pod  1/1     Running   10.244.1.7   demo-cluster-worker
```

└──────────┘ └──────────────────────────┘
the POD's own IP always a WORKER,
 never the control plane

✓ "Whatever we are deploying, it is going into the worker node, not the control plane, not the master node." That's §3's rule — the control plane runs no applications — visible in one column.

🤔 The localhost problem, again

😊 CLASS MOMENT

She couldn't open `localhost:80` in her browser. "Again, it's a Mac issue or what? I'm not able to see nginx. In my laptop also not working."

✓ **Her ruling, and it's the right one:** "Even if you're not able to see it in your browser, that's okay. If you're able to see `1/1`, that means your nginx application is working, fine."

🧩 **Why the browser doesn't work, since "Mac issue" isn't the real reason:** your pod's IP (`10.244.x.x`) lives **inside the cluster's private network**, which is itself inside a Docker container. `localhost:80` on your laptop isn't connected to it at all. Nothing is broken — you simply haven't built the bridge yet.

The bridge, which you can use right now:

```
kubectl port-forward pod/nginx-pod 8080:80
# then open http://localhost:8080 ✓
```

Class 11's **Service + nodePort** is the permanent version of this.



11



The Pod Lifecycle — And The Three-Stage Race

This was the best segment of the class.



SAID IN CLASS

"Same as the container lifecycle we learned in Docker. Same thing we have here."



```
ContainerCreating → Running → Completed ✓ the task finished
                        |
                        → Failed
                        |
                        ErrImagePull / ImagePullBackOff ✗ bad image name
```



Lab 2 — catch all three stages (the race)

She set this as a live challenge, and one student won it.

DO THIS YOURSELF

Lab 2 — the busybox race.

Step 1 — `hello.yml` :

```
apiVersion: v1
kind: Pod

metadata:
  name: hello-pod

spec:
  restartPolicy: Never
  containers:
  - name: hello
    image: busybox
    command: ["sh", "-c", "echo Hello Kubernetes"]
```

Step 2 — apply it, then spam `get pods` as fast as you can type:

```
kubectl apply -f hello.yml
kubectl get pods      # ContainerCreating
kubectl get pods      # Running           ← the hard one to catch
kubectl get pods      # Completed
```

Can you catch all three?

CLASS MOMENT

How it went in class. Most people caught only two — `ContainerCreating` and `Completed` . Then:

"One person got three of three stages. He's a fast typer. By three commands he's able to see container creating, running and completed. Three stages. He's sending the screenshot."

And her attempt: *"What a story — you can fast type, I'm not. I'm just typing here and it's going to Completed."* Then, defensively: *"My laptop is fast, by the way. It's not my fault, my laptop is actually really fast."* 😊

When the screenshot came through: *"Clap for Manav. In milliseconds, in microseconds, he used this command."*

💡 The concept the race is actually teaching

🎯 INTERVIEW ALERT

She asked: "nginx stays **Running** forever. busybox goes to **Completed** in seconds. Why?"

```
nginx —
no `command:` → runs the image's default
→ nginx server, designed to run FOREVER
→ status stays RUNNING ✓

busybox —
command: ["sh","-c","echo Hello Kubernetes"]
→ prints one line → the process EXITS
→ task done → status COMPLETED ✓
```

🗣️ "For nginx, we want it to run always — that's why it will not go to Completed. For busybox we specifically mentioned a command: you create a container, you run this command, and your task is done. Task done means the container will be stopped, so it goes to Completed."

✓ **It's Docker's PID 1 rule again (Class 6 §13).** A container lives exactly as long as its main process. And **command:** in Kubernetes is the same thing as **CMD** in a Dockerfile — she said so directly: "Do you know in Docker we also added CMD? It's the same thing."

⚠️ WATCH OUT

And this is why **restartPolicy: Never matters here.** Without it the default is **Always**, so Kubernetes would see the container exit, restart it, watch it exit again — forever. You'd get **CrashLoopBackOff** on a pod that is working perfectly. **Never** tells Kubernetes "finishing is the point."

🧪 Lab 3 — break it on purpose

She did this live, deliberately.

DO THIS YOURSELF

Lab 3 — cause an `ErrImagePull` .

Step 1 — put a nonsense image name in `hello.yml` :

```
- name: hello
  image: abcd          # ← mash the keyboard
```

Step 2 — apply and look:

```
kubectl apply -f hello.yml
kubectl get pods
# NAME          READY   STATUS             RESTARTS   AGE
# hello-pod     0/1     ErrImagePull       0          5s
#
#               ↓ a few seconds later
# hello-pod     0/1     ImagePullBackOff   0          30s
```

Step 3 — find out why:

```
kubectl describe pod hello-pod    # scroll to Events at the bottom
```

INTERVIEW ALERT

She flagged this as a fresher interview question:

"If you add any random image inside your container — will the container be created or not?"

✅ **The answer: the POD is created. The CONTAINER is not.**

"As you know, the pod is created right now. I did not face any issue here — 'hello pod is created.' But when I do `kubectl get pods` , I can see `ErrImagePull`, because the image itself is not there. There is no image like this on Docker Hub."

🧩 **Why the two-step:** `kubectl apply` only writes your object into **etcd** — and "write a row in a database" always succeeds. The *pulling* happens later, on the node, by kubelet. So creation and starting are two different moments, and only the second one can fail this way.

💡 **And `ErrImagePull` → `ImagePullBackOff` ?** Kubernetes retries, and each retry waits longer than the last (back-off) so it doesn't hammer the registry forever.



12 minikube vs kind vs kubectl vs EKS

She cleared up confusion that came from a student seeing `control-plane` instead of `worker` in `-o wide`.

	What it is	Where it runs
minikube	a local cluster — "the best thing for a beginner"	your laptop
kind	a local cluster — "Kubernetes IN Docker". "Very limited resources. You need to understand how to use kind."	your laptop
kubectl	⚠️ NOT a cluster — "kubectl is a CLI that we downloaded"	your laptop
EKS / AKS / GKE	a cloud cluster	a cloud provider

⚠️ WATCH OUT

The distinction she repeated: "kubectl is not a cluster. kubectl is a CLI." It's the remote control; minikube/kind/EKS are the television.

😊 CLASS MOMENT

Her own setup: "I have three clusters locally — one kind, one minikube, and kubectl too. Right now the kind cluster is already running. I set it in my cron job — every morning at 4 or 5 a.m. it will run the kind cluster."

💡 That's Class 3's cron, used in anger. 😊

✅ LOCK THIS IN

For this course: use minikube. "I also started from minikube. `minikube start` and `minikube stop` — it's a very, very easy command to run."



13 The Command Set

Every command she ran, grouped by what it replaces in Docker.

Docker	Kubernetes
<code>docker run -dit -p 80:80 nginx</code>	<code>kubectl apply -f pod.yml</code>
<code>docker ps</code>	<code>kubectl get pods</code>

Docker	Kubernetes
<code>docker logs <id></code>	<code>kubectl logs <pod></code>
<code>docker inspect <id></code>	<code>kubectl describe pod <pod></code>
<code>docker rm <id></code>	<code>kubectl delete pod <pod></code>
<code>docker exec -it <id> sh</code>	<code>kubectl exec -it <pod> -- sh</code>
(no equivalent)	<code>kubectl get nodes</code>

Cluster & setup

```

minikube start
minikube status
minikube stop
kubectl version
kubectl cluster-info
kubectl get nodes

```

Working with pods

```

kubectl apply -f pod.yml      # create OR update
kubectl get pods             # list
kubectl get pods -o wide     # + IP and NODE
kubectl logs <pod>           # container output
kubectl describe pod <pod>   # everything, incl. Events ← debugging
kubectl delete pod <pod>     # remove
kubectl port-forward pod/<pod> 8080:80 # reach it from your browser

```

💡 GOOD TO KNOW

apply vs create — she touched on this when a student's second `apply` didn't error:

"If you run the command again it will give you an error that it already exists. But if you run `apply`, it will override whatever you have. And if it's not there, it will create it."

```

kubectl create -f x.yml → ❌ errors if it already exists
kubectl apply -f x.yml → ✅ creates it, or updates it. Run it 100 times.

```

✅ **Always use `apply`**. That's what "declarative" means in practice — you describe the end state, and running it twice is harmless.



14 Homework

Task

- 1 ⚠️ **INSTALL MINIKUBE.** Non-negotiable — "if it's not there, get out" 😊
- 2 🖋️ **Run every YAML file** in her repo once: `deployment.yml`, `replicaset.yml`, `statefulset.yml`, `daemonset.yml` — and **watch what happens** with `kubectl get pods`
- 3 📖 **Learn what each one means:** what is a DaemonSet? a Deployment? a replica? a StatefulSet — and **where would you use one?**
- 4 🏆 Redo **Labs 1–3** above and screenshot the results into your README
- 5 🎯 Try to catch **all three pod stages** (the race)
- 6 📖 Read `architecture.md` and `core-objects.md` in her [Kubernetes repo](#)

📌 NOTE

Her preview of next class: Services, and "taint and toleration, and affinity — we're going to learn it in the next lectures, intermediate Kubernetes."



15 Cheat-Sheet

The four fields

```
apiVersion: # v1 (core) or apps/v1 (manages pods)
kind:       # Pod, Service, ReplicaSet, Deployment, StatefulSet, DaemonSet
metadata:   # name, labels
spec:      # the actual configuration
```

apiVersion lookup

Kind	apiVersion
Pod, Service, Namespace, ConfigMap, Secret	v1
Deployment, ReplicaSet, StatefulSet, DaemonSet	apps/v1

Pod statuses

Status	Means	Do this
<code>ContainerCreating</code>	pulling the image, starting up	wait
<code>Running</code>	✅ alive	—
<code>Completed</code>	finished successfully	expected with a <code>command</code> + <code>restartPolicy: Never</code>
<code>Failed</code>	exited non-zero	<code>kubectl logs</code>
<code>ErrImagePull</code>	can't fetch the image	⚠️ check the image name
<code>ImagePullBackOff</code>	given up retrying (for now)	same
<code>CrashLoopBackOff</code>	starts, dies, restarts, repeats	<code>kubectl logs</code> , or you need <code>restartPolicy: Never</code>
<code>Pending</code>	not scheduled	<code>kubectl describe</code> — usually no node has the resources

Concepts

Term	Means
API object	everything in Kubernetes. Delete the object → the thing is gone
Pod	smallest deployment unit; shared network + volume
Sidecar / agent	helper container in the same pod (logs, security — e.g. Falco)
ReplicaSet controller	keeps N pods alive
Node controller	checks the servers are alive
kubelet	runs containers + sends the heartbeat
kube-proxy	networking on a node
CRI / containerd	the engine that actually runs containers
The golden rule	🔑 no component talks to another directly — everything through the API server
minikube / kind	local clusters
kubectl	⚠️ the CLI, not a cluster
<code>command:</code>	same as Docker's <code>CMD</code>
<code>restartPolicy: Never</code>	for pods that are <i>meant</i> to finish





- **Instructor name:** the header credits "*Ritesh Prajapati*" — that's the **Scaler++ Chrome extension's developer**, not the teacher. The instructor is **Nensi Ravaliya** ([@Nency-Ravaliya](#)).
- **This replaces an earlier reconstructed version of these notes.** Class 10 previously had no transcript, so its notes were rebuilt from the YAML files and her repo. **This version is written from the actual recording** — so the architecture walkthrough (§4), the pod-lifecycle race (§11) and the ErrImagePull demo (§11) are what she really said and did, not inference.
- **Roughly the middle 20 minutes of the recording is unusable** — a long stretch of "*Thank you*", Portuguese and Spanish filler while she walked the room checking laptops one by one. No teaching happened in it; the class resumes at the repo check-ins.
- 🔍 **Two clarifications added** rather than silent rewrites: **containerd was extracted from Docker** rather than replacing it (§5), and **localhost:80 failing is cluster networking, not "a Mac issue"** (§10) — with `kubectl port-forward` given as the fix she didn't mention.
- Garbled terms corrected: "ATCD / PTCD" → `etcd`, "QBlit / Qblit / Kubelet" → `kubelet`, "Qproxy / Q-proxy" → `kube-proxy`, "QBAPI server" → `kube-apiserver`, "Vaucon node" → `worker node`, "Swamp cluster" → `Docker Swarm`, "quad.yaml" → `pod.yaml`, "Qtctl / ctl" → `kubectl`, "Minicube / Many kubers" → `minikube`, "call host" → `localhost`, "Geek4Geeks" → `GeeksforGeeks`, "demon set" → `DaemonSet`, "image error pool / error image pull" → `ErrImagePull`, "CK administrator" → `Kubernetes administrator`.